# Week 12
# RL for Ranking and Two-Sided Matching

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

# Contents


**Learning objectives** 4

**1 Motivation and intuition** 5
- 1.1 The applied stack . . . 5
- 1.2 Anchoring . . . 6

**2 Mathematical formulation** 7
- 2.1 The ranking task . . . 7
- 2.2 Counterfactual Learning-To-Rank . . . 7
- 2.3 Slate problems and SlateQ . . . 8
- 2.4 REINFORCE-Top-K with off-policy correction . . . 8
- 2.5 Fairness . . . 9
- 2.6 Calibration . . . 9
- 2.7 Cold-start and meta-learning bandits . . . 10
- 2.8 Calibration under distribution shift . . . 10
- 2.9 Two-sided markets and stability . . . 11
- 2.10 Incentive compatibility and Myerson . . . 11
- 2.11 The supply-side equilibrium hazard . . . 11

**3 Deep dive: the synthetic insurance / ATS reference architecture** 12
- 3.1 Data contract . . . 12
- 3.2 Latency budget . . . 12
- 3.3 Failure modes and tests . . . 13
- 3.4 Logging . . . 13

**4 Two-tower retrieval: the dense-encoder revolution** 14
- 4.1 The retrieval problem . . . 14
  - 4.1.1 Why “two towers”? . . . 14
- 4.2 Contrastive learning . . . 14
  - 4.2.1 In-batch vs. mined negatives . . . 15
- 4.3 Domain adaptation . . . 15
- 4.4 Evaluation: NDCG, MRR, Recall@k . . . 15

**5 Cross-encoder re-ranking: when latency permits** 16
- 5.1 Why we still need the bandit on top . . . 16

**6 Algorithms** 17

**7 Implementation walkthrough** 19
- 7.1 The two-tower . . . 19
- 7.2 The LinTS reranker . . . 19
- 7.3 The fairness audit . . . 20
- 7.4 Beta calibration in three lines . . . 20
- 7.5 The end-to-end pipeline . . . 20

**8 Bayesian derivations for the LinTS reranker** 22
- 8.1 The posterior in closed form . . . 22

8.2 Posterior sampling . . . 22
8.3 Online posterior updates . . . 22
8.4 Confidence ellipsoids vs. posterior samples . . . 23
8.5 Pitfalls when scaling to deep features . . . 23

**9 Worked numerical examples 24**
9.1 Worked example 1: a 4x3 two-tower . . . 24
9.2 Worked example 2: IPS-LTR weighting . . . 24
9.3 Worked example 3: Beta calibration . . . 24

**10 Empirical results 25**

**11 Reading published deployments critically 28**
11.1 LinkedIn ranking constraints . . . 28
11.2 YouTube policy learning and slate values . . . 28
11.3 A reproducible transfer exercise . . . 28

**12 Safety angle 29**

**13 Matching application 30**

**14 Production deployment checklist 32**
14.1 Data . . . 32
14.2 Model . . . 32
14.3 Online infrastructure . . . 32
14.4 Logging . . . 32
14.5 Audit . . . 33
14.6 Compliance . . . 33

**15 Exercises 34**

**16 Further reading 35**

# Learning objectives

**Study companion.** Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

**Key Idea**

This is the **flagship applied lecture** of the curriculum. By the end of this week you will be able to:

1. Derive the *position-bias* click model and the IPS-LTR / SNIPS-LTR estimators (Joachims, Swaminathan, Schnabel WSDM 2017).
2. Decompose a slate Q-value under the SlateQ single-choice user model (Ie et al. 2019) and derive the REINFORCE-Top-K off-policy correction $\lambda_K(a|s) = K(1-\pi(a|s))^{K-1}$ (Chen et al. WSDM 2019).
3. Train a two-tower retriever for CV $\leftrightarrow$ JD matching, then bolt a LinTS contextual-bandit re-ranker on top.
4. State and audit *three* fairness criteria: demographic parity, equalised opportunity, and exposure-fairness (Singh & Joachims 2018).
5. Calibrate a binary classifier with Platt, isotonic, and Beta calibration; compute ECE and Brier; read off a reliability diagram.
6. Reason about a two-sided market: stability (Gale–Shapley), competing bandits (Liu–Mania–Jordan AISTATS 2020), and incentive compatibility.
7. Connect the entire pipeline to the hypothetical ATS architecture: retrieval $\rightarrow$ cross-encoder $\rightarrow$ bandit reranker $\rightarrow$ fairness constraints $\rightarrow$ logged decisions with propensities $\rightarrow$ DR-OPE.

# 1 Motivation and intuition

This is where the curriculum's two domains — synthetic insurance lead matching and synthetic ATS candidate ranking — collide with the bandit + OPE machinery of Week 11. The change of scenery: *the action space is ordered.* We do not pick one broker or one candidate; we present a ranked list and the user (recruiter, broker) only interacts with the top of the list. Two problems follow:

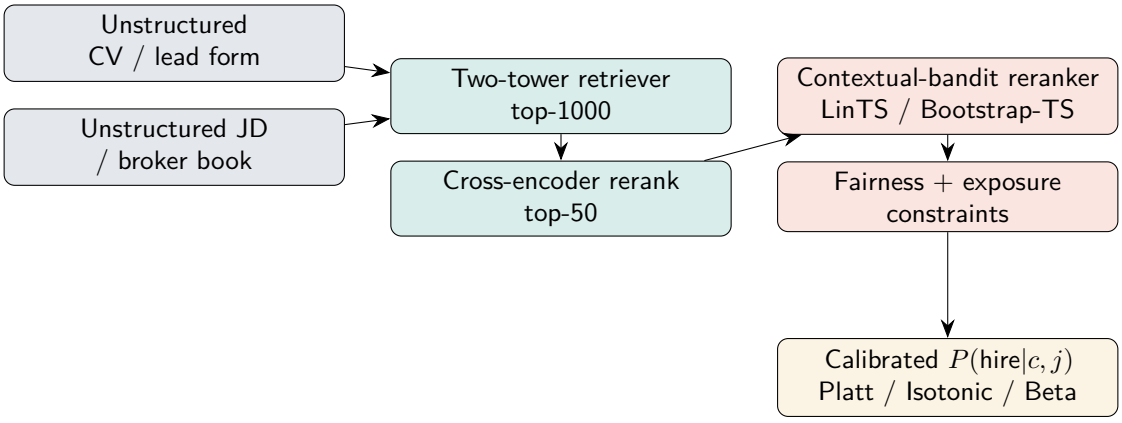
**Position bias.** A click on the candidate at rank 1 carries qualitatively different information from a click at rank 50. A candidate at rank 50 was likely never seen. Naive "the model that maximises CTR" training reduces to "the model that orders by past CTR", producing a rich-get-richer feedback loop. The fix is to model the examination process explicitly — the position-bias model of Richardson, Dominowska, Ragno 2007 — and to importance-weight clicks by examination probability (Joachims, Swaminathan, Schnabel WSDM 2017).

**Two-sidedness.** Both synthetic insurance (broker ↔ lead) and ATS (candidate ↔ job) are two-sided markets. Brokers reject leads they cannot service. Candidates reject jobs they do not want. Recruiters reject candidates whose CV is mis-formatted. A unilateral recommender that maximises one side's reward loses to a recommender that respects equilibrium constraints: stable matchings (Gale–Shapley 1962), incentive-compatible mechanisms (Roth 1984; Hatfield–Milgrom 2005), exposure-fairness (Singh & Joachims 2018).

**Calibration.** A miscalibrated $P(\text{hire} \mid c, j)$ corrupts every downstream decision: shortlist thresholds, send-to-recruiter auto-approvals, fairness audits, revenue forecasts. Calibration is a free post-processing step that pays for itself a thousand times over.

## 1.1 The applied stack

The 2025 canonical applied stack is a three-layer cascade:




The mini-project for this week builds a CPU-tractable miniature of this stack on a synthetic 200-candidate / 50-job ATS dataset. The two-tower is a two-layer MLP trained with in-batch-negatives InfoNCE. The reranker is LinTS over the candidate skill bag plus the two-tower score. The fairness audit measures demographic parity, equalised opportunity, and exposure-by-group on a gender-coded subset. The calibration step compares Platt, isotonic, and Beta on a held-out hire-outcome log.

## 1.2 Anchoring

If you have built a search engine you have built a ranker. If you have built a recommender you have built a slate-bandit. If you have built a classifier you have a calibration problem, even if you do not know it yet. The new material is (i) the position-bias correction (clicks are not labels), (ii) the exposure-fairness primitive (rank tail matters), and (iii) the LinTS reranker (uncertainty over candidate quality given a job).

**Common pitfall**

"We will A/B test it" is a slogan, not a methodology. Recruiter-side and broker-side A/B tests have $n_{\text{eff}} = O(\text{recruiters})$, not $O(\text{searches})$ — because outcomes within a recruiter are correlated. Plan for OPE-as-default, A/B-test-as-confirmation.

# 2 Mathematical formulation

## 2.1 The ranking task

**Definition 2.1.1** (Ranking with implicit feedback)**.** A user $u$ issues a query $q$. The system returns an ordered list $\pi(q) = (a_1, a_2, \ldots, a_K)$ of items. The user examines item at position $k$ with probability $p_k$ (the position bias); if examined, clicks with probability $\rho(a_k, q)$ (the relevance). The reward is the click vector $c \in \{0,1\}^K$ with $c_k \sim \mathrm{Bern}(p_k \cdot \rho(a_k, q))$.

The factorisation $\mathbb{P}(c_k = 1) = p_k \cdot \rho(a_k, q)$ is the *Position-Based Click Model* (PBM; Richardson, Dominowska, Ragno 2007). More expressive models (cascade, DBN, NCM) allow $p_k$ to depend on the ranking and earlier clicks; the PBM is the simplest version that already captures the qualitative effect.

**Definition 2.1.2** (Ranking metric: NDCG@k)**.** Given relevances $\mathrm{rel}_1, \ldots, \mathrm{rel}_K$ at positions $1, \ldots, K$,

$$\mathrm{DCG@}K(\pi) = \sum_{k=1}^{K} \frac{\mathrm{rel}_k}{\log_2(k+1)}, \qquad \mathrm{NDCG@}K(\pi) = \frac{\mathrm{DCG@}K(\pi)}{\mathrm{IDCG@}K},$$

where the ideal IDCG@$K$ is DCG@$K$ of the relevance-sorted ranking. NDCG@$K \in [0,1]$.

## 2.2 Counterfactual Learning-To-Rank

**Theorem 2.2.1** (IPS-LTR unbiasedness)**.** *For a fully-ranked, examination-model PBM, let $c_k^{(\pi_b)}$ be the observed click on position $k$ of the logger's ranking $\pi_b(q)$. The estimator*

$$\hat{U}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \sum_{k : a_k^{(\pi_b)} \in \pi(q_i)} \frac{c_k^{(\pi_b)}(q_i)}{p_{k_b(i)}}$$

*is unbiased for $U(\pi) = \mathbb{E}_q \sum_a \mathbb{1}\{a \in \pi(q)\}\rho(a,q)$ (the "relevance reach").*

*Sketch.* For each query $q_i$ and item $a$ shown at position $k_b$ by $\pi_b$, $\mathbb{E}[c_{k_b}^{(\pi_b)}] = p_{k_b}\rho(a, q_i)$. Dividing by $p_{k_b}$ removes the position bias: $\mathbb{E}[c_{k_b}^{(\pi_b)}/p_{k_b}] = \rho(a, q_i)$. Restricting to items also in $\pi(q_i)$ and summing recovers $\sum_a \mathbb{1}\{a \in \pi(q_i)\}\rho(a, q_i)$. □ □

*Intuition.* You can read IPS-LTR as: "each click is worth more if the user could barely see the item it was on". A click at rank 50 is much more informative about relevance than a click at rank 1.

The variance of IPS-LTR explodes when $\pi_b$ shows low-relevance items at deep ranks (small $p_k$, near-zero clicks, huge importance weights). SNIPS, DR, and propensity-clipping all carry over from Week 11. The Joachims line of work (2017–2024) primarily refines this estimator: dual learning (Ai et al. 2018) jointly estimates $p_k$ and $\rho$; cascade-CLTR (Vardasbi et al. 2020) handles non-PBM models; the 2023 DR-LTR (Oosterhuis SIGIR 2023) adds a direct-method baseline.

## 2.3 Slate problems and SlateQ

A user does not see one item; they see a slate $\mathbf{A} = (a_1, \dots, a_K)$. The Q-function over slates has cardinality $|\mathcal{A}|^K$ — intractable. SlateQ (Ie et al. IJCAI 2019) factorises under the *single-choice* user model: at each step the user picks exactly one item from the slate according to a known choice rule $P(a \mid \mathbf{A}, s)$ (e.g., softmax, multinomial logit). Then

**Proposition 2.3.1** (SlateQ decomposition: assumptions matter)**.** *Assume at most one selected item, allow a null-selection outcome, and assume reward and next-state dynamics depend on the slate only through that selected item. Then*

$$Q^\pi(s, \mathbf{A}) = \sum_{a \in \mathbf{A} \cup \{\varnothing\}} P(a \mid \mathbf{A}, s)\, \bar{Q}^\pi(s, a).$$

*The item value conditions on selection, with future slates following $\pi$. This follows by total expectation. A tractable representation does not make arbitrary slate optimisation easy; the paper's efficient optimisation results need additional choice-model structure.*

Source: https://arxiv.org/abs/1905.12767.

*Remark.* The single-choice assumption fails when users browse multiple items in a slate (e.g., a recruiter scrolling through 10 CVs before contacting any). Choose or validate a multi-choice model if several displayed items can affect reward or continuation. Small slate size alone does not make the single-choice assumptions true.

## 2.4 REINFORCE-Top-K with off-policy correction

The WSDM 2019 paper by Chen, Beutel, Covington, Jain, Belletti and Chi describes an industrial REINFORCE recommender. The setup: a softmax policy $\pi_\theta(a \mid s) \propto \exp f_\theta(s, a)$ over millions of videos; log a slate of $K$ (e.g. 50) recommendations; observe which one was played; train.

The vanilla policy gradient gives a high-variance, off-policy-biased estimator. Chen et al. stabilise it via three tricks:

**(1) Importance sampling correction.** The behaviour policy $\beta(a \mid s)$ is the historical production policy; the new policy $\pi(a \mid s)$ is what we are training. The off-policy policy gradient is

$$\nabla_\theta J(\pi) \approx \mathbb{E}_{(s,a,r)\sim\beta}\Big[\frac{\pi_\theta(a \mid s)}{\beta(a \mid s)} \nabla_\theta \log \pi_\theta(a \mid s) \cdot R(s, a)\Big].$$

**(2) Clipping.** Capping importance weights limits individual weighted contributions when rewards and scores are bounded, but introduces bias. This differs from PPO's minimum of clipped and unclipped surrogates: PPO does not simply substitute capped weights in every gradient term.

**(3) Top-K correction.** The behaviour policy presents a slate of $K$ items, not 1. The probability that item $a$ appears after $K$ independent with-replacement draws is exactly $1 - (1 - \pi(a \mid s))^K$. Deduplication can give fewer than $K$ items; another slate sampler needs its own inclusion probability. The gradient correction factor is the derivative w.r.t. $\pi(a \mid s)$:

**Theorem 2.4.1** (Top-K correction, Chen et al. WSDM 2019)**.** *Let $\alpha_K(a \mid s) = 1 - (1 - \pi(a \mid s))^K$. Then*

$$\frac{\partial \alpha_K}{\partial \pi} = K(1 - \pi(a \mid s))^{K-1} \triangleq \lambda_K(a \mid s),$$

*and the top-K policy gradient is*

$$\nabla_\theta J_K(\pi) \approx \mathbb{E}_{(s,a,r)\sim\beta}\Big[\frac{\pi_\theta(a \mid s)}{\beta(a \mid s)} \cdot \lambda_K(a \mid s) \cdot \nabla_\theta \log \pi_\theta(a \mid s) \cdot R(s, a)\Big].$$

*Proof.* Independently draw $K$ items *with replacement*, then deduplicate. The probability of never drawing $a$ is $(1-\pi(a \mid s))^K$. For an item-additive inclusion objective $J_K = \sum_a \alpha_K(a \mid s)R(s,a)$, differentiate: $\nabla J_K = \sum_a \lambda_K(a \mid s)R(s,a)\nabla\pi(a \mid s)$. Reweight this sum by a supported logger to obtain the displayed gradient. The final set may have fewer than $K$ items. A fixed-size without-replacement sampler needs its own inclusion probability. See Chen et al., Section 4.3, https://arxiv.org/abs/1812.02353. □

*Intuition.* $\lambda_K(a \mid s) = K(1-\pi)^{K-1}$ down-weights the gradient on items that $\pi$ is already confident about (because they will be in the top-K anyway), and up-weights items that $\pi$ is uncertain about (because that is where the marginal gain of training lives).

## 2.5 Fairness

**Definition 2.5.1** (Demographic parity (DP))**.** A ranker satisfies DP for a sensitive attribute $A$ if $\mathbb{P}(a \in \text{top-}k \mid A = 0) = \mathbb{P}(a \in \text{top-}k \mid A = 1)$.

**Definition 2.5.2** (Equalised opportunity (EO), Hardt et al. 2016)**.** A ranker satisfies EO if $\mathbb{P}(a \in \text{top-}k \mid A = 0, \text{qualified}) = \mathbb{P}(a \in \text{top-}k \mid A = 1, \text{qualified})$, where "qualified" is a task-specific binary indicator (e.g., true match $\geq \tau$).

**Definition 2.5.3** (Exposure fairness, Singh & Joachims 2018)**.** For a ranking $\pi$ and position-bias vector $p$, the *exposure* of item $a$ is $\exp(a) = \sum_k \mathbb{1}\{\pi_k = a\}p_k$. A group-fair ranker requires

$$\frac{\mathbb{E}[\exp(a) \mid A = 0]}{\mathbb{E}[\exp(a) \mid A = 1]} \approx 1 \quad (\text{"equal exposure"}).$$

*Intuition.* DP says "equal representation in the top-k". EO says "equal representation *among the qualified*" — you do not have to promote unqualified candidates, you just have to not skip past qualified ones. Exposure fairness says "equal time on the user's screen" — which is a producer-side fairness criterion (the brokers / candidates being ranked).

DP is the easiest to enforce (a simple re-ranker can equalise top-k proportions). EO is harder because it requires a ground-truth qualification label. Exposure fairness is the dominant production criterion for two-sided markets because it directly maps to the unit of value: attention.

*Remark* (FairCo is a feedback controller). Morik et al. (SIGIR 2020) use a proportional correction based on accumulated merit-normalised exposure or impact disparity. FairCo is not the linear-program baseline it is compared against. Its convergence statement needs the paper's feasibility and estimation assumptions; there is no general $O(\epsilon)$ NDCG-loss guarantee for every fairness tolerance. Source: https://arxiv.org/abs/2005.14713.

## 2.6 Calibration

**Definition 2.6.1** (Calibration)**.** A predictor $f : \mathcal{X} \to [0,1]$ is *calibrated* if $\mathbb{P}(Y = 1 \mid f(X) = p) = p$ for almost every $p \in [0,1]$.

**Definition 2.6.2** (ECE)**.** The Expected Calibration Error binned with $M$ bins is

$$\text{ECE}(f) = \sum_{m=1}^{M} \frac{|B_m|}{n} \cdot |\text{acc}(B_m) - \text{conf}(B_m)|.$$

**Theorem 2.6.3** (Platt scaling, Platt 1999)**.** *Platt scaling fits $f_{\text{cal}}(x) = \sigma(a \cdot f(x) + b)$ by maximum likelihood on a held-out set, where $\sigma$ is the logistic function. This is logistic regression on a single feature (the model's score) and is the most-used calibration baseline.*

**Theorem 2.6.4** (Isotonic regression, Zadrozny & Elkan 2002)**.** *Fit a monotone non-decreasing piecewise-constant function* $g : [0,1] \to [0,1]$ *minimising* $\sum_i (g(f(x_i)) - y_i)^2$*, by the pool-adjacent-violators algorithm in* $O(n)$ *after scores are sorted. This fits monotone block averages, not every individual binary label. Held-out behaviour depends on sample size and distribution; there is no universal 1000-row threshold.*

**Definition 2.6.5** (Beta calibration, Kull, Filho, Flach 2017)**.** For $s \in (0,1)$, use $f_{\text{cal}}(s) = \sigma(a \log s - b \log(1-s) + c)$. Nonnegative $a, b$ yield a non-decreasing map, and $a = b = 1, c = 0$ recovers the identity. If $a = b$, the map is logistic in the *logit* of $s$, not Platt scaling on the raw probability $s$.

The local logistic-regression fit does not constrain its coefficients, so check monotonicity when that property is required. Fit on a calibration split and compare held-out Brier score, log loss and reliability curves; no calibrator universally dominates another. Source: https://proceedings.mlr.press/v54/kull17a.html.

## 2.7 Cold-start and meta-learning bandits

A brand-new requisition has zero history. A flat LinTS on this requisition has flat posterior $\mathcal{N}(0, v^2 I)$ — every candidate is equally likely to be selected. This is uninformative and burns recruiter trust.

The fix is hierarchical Thompson Sampling (Kveton et al. ICML 2021): each requisition's prior $\theta_j \sim \mathcal{N}(\mu_C, \Sigma_C)$ is drawn from a category-level distribution (e.g., "data-engineer requisitions"), and the category distribution itself has a hyper-prior. New requisitions inherit the category posterior; over time the per-requisition posterior overrides it.

*Remark* (What meta-learning can establish)*.* MetaTS learns a prior across related bandit tasks. Its Gaussian-bandit Bayes-regret analysis depends on the task prior and learning protocol. Do not compare invented dimension-free rates or infer improvement merely by adding a cross-task term to an independent-task bound. Source: https://proceedings.mlr.press/v139/kveton21a.html.

## 2.8 Calibration under distribution shift

Calibration is a property of $(P_X, f)$; it fails when $P_X$ shifts. ATS distribution shifts:

- New geographic locale (different skill base rates).
- New requisition type (new salary band, new seniority).
- Seasonal effect (graduate hiring in summer).
- Calibration set staleness (last week's labels are not this week's).

*Remark* (Calibration under distribution shift)*.* Calibration on a source population need not survive a changed population. Importance weighting can identify target expectations under covariate shift with unchanged $P(Y \mid X)$ and appropriate density-ratio support. It does not repair arbitrary conditional-label changes. Even with the same $P(Y \mid X)$, a coarse score's calibration can change when the mixture of inputs within score bins changes. This is a scope warning, not a universal theorem that one reweighting recipe fixes every shift.

**Mitigations:**

- *Stratified calibration*: fit a separate calibrator per slice (per locale, per requisition family). Reduces capacity per slice but does not eliminate shift within a slice.
- *Online recalibration*: refit Beta / isotonic on a sliding-window calibration set; bound the drift in ECE by a target.
- *Drift monitors*: KL divergence of this-week's score distribution against last-month's; alert if exceeded.

## 2.9 Two-sided markets and stability

**Definition 2.9.1** (Stable matching, Gale–Shapley 1962)**.** A matching $\mu$ of $n$ candidates to $m$ jobs (with each side ranking the other) is *stable* if there is no pair $(c, j)$ such that $j \succ_c \mu(c)$ and $c \succ_j \mu(j)$. The deferred-acceptance algorithm produces a stable matching in $O(nm)$ time and is candidate-optimal when candidates propose.

*Remark* (Learning preferences in a matching market)*.* Liu, Mania and Jordan (AISTATS 2020) study players learning their reward preferences while arms have preferences over players. Only one side's preferences are learned in this model. Stable-regret results depend on centralised versus decentralised matching and on the stable comparator; they are not a generic $\widetilde{O}(K\sqrt{T})$ guarantee when both sides learn arbitrary strategies. Source: https://proceedings.mlr.press/v108/liu20c.html.

## 2.10 Incentive compatibility and Myerson

**Definition 2.10.1** (Dominant-strategy incentive compatibility (DSIC))**.** A mechanism is DSIC if reporting truthful preferences is a dominant strategy for every agent, regardless of others' reports.

*Remark* (Revelation principle)*.* An outcome implementable in a specified equilibrium concept can be represented by a truthful direct mechanism with the corresponding incentive constraints. Dominant-strategy implementation gives a DSIC direct mechanism; Bayesian implementation gives Bayesian incentive compatibility. Arbitrary implementability does not imply DSIC.

*Remark* (Myerson allocation: scope)*.* For independent private values and regular value distributions in a single-item auction, allocate to the highest nonnegative virtual value $\phi_i(v) = v - (1 - F_i(v))/f_i(v)$ and use the corresponding payments. The seller may withhold the item. Irregular distributions require ironing; correlated values or externalities change the problem.

**Why this matters for synthetic insurance / ATS.** If brokers bid (or set referral fees) and the platform allocates leads, the platform is running an auction. A first-best CTR-maximising auction is not Myerson-optimal in revenue and is often not even DSIC. The matching layer must respect mechanism-design constraints — otherwise brokers learn to game it (under-bidding, lead-poaching, lead-rejection cascades).

## 2.11 The supply-side equilibrium hazard

**Modelling question.** When producers adapt to the platform's objective, evaluation should include that response. A static ranking result alone does not establish the existence, uniqueness or welfare of a strategic equilibrium. Analyse a specified game before claiming a particular objective preserves producer diversity.

This is the formal version of "you get what you measure". A synthetic insurance allocator that rewards bind rate pushes brokers to specialise in easy leads; a recruiter system that rewards interview-rate pushes recruiters to chase easy candidates. The platform's objective shapes the supply side.

# 3 Deep dive: the synthetic insurance / ATS reference architecture

This chapter recapitulates and extends the cascade from the motivation chapter into the level of detail an engineer can actually implement against. We cover, layer by layer: the data contract (what each layer expects), the latency budget, the failure modes specific to that layer, and the production tests that catch them.

## 3.1 Data contract

Every layer in the cascade has a typed input/output contract. The contract is part of the model artefact (stored alongside weights). A version mismatch between two layers is the most common production incident.

| Layer | Input | Output |
|---|---|---|
| Parse | raw CV / quote form | structured `Candidate` / `Lead` object |
| Two-tower | `Candidate.skills` (or text) | $d_e$-dim L2-normalised embedding |
| Cross-encoder | (CV-text, JD-text) | relevance scalar in $[0, 1]$ |
| Bandit reranker | top-50 candidate features + JD features | exposure scores |
| Fairness LP | exposure scores + sensitive attrs | adjusted exposure scores |
| Calibrator | adjusted scores | $P(\text{hire} \mid c, j)$ |

## 3.2 Latency budget

A production ATS shortlist call has a budget of $\sim$1 second (recruiter clicks "next candidate" once per second). The breakdown:

- Parse: 5–20 ms (one-off, cached).
- Two-tower retrieval: 1–10 ms (FAISS index of $10^5$–$10^7$ candidates).
- Cross-encoder rerank: 200–800 ms (top-1000 $\rightarrow$ top-50).
- Bandit reranker: 1–5 ms (linear-algebra on top-50 features).
- Fairness LP: 10–50 ms (small LP on top-50).
- Calibrator: $<$1 ms (parametric Beta / isotonic lookup).

The cross-encoder is the latency-dominant layer. Production tricks: batch candidates, validate quantisation and reduce the candidate set. In a bidirectional cross-encoder, JD representations attend to the candidate; an ordinary autoregressive prefix KV cache cannot be reused unchanged.

## 3.3 Failure modes and tests

**Parse layer.** CV parsing fails silently when CV structure changes (PDF watermarks, two-column layouts, scanner artefacts). Test: a "shadow-parse" pipeline that reads CVs into a strict schema and alerts when the empty-field rate exceeds a threshold.

**Two-tower layer.** Embedding drift: the encoder is re-trained, the index is not rebuilt, scores are wrong. Test: per-build embedding-version hash; reject queries when the index version mismatches the encoder version.

**Cross-encoder layer.** Inference OOM under burst load; latency spikes; queue overflows. Test: per-pod token-rate limit; load-shed by serving bi-encoder-only when the cross-encoder is saturated.

**Bandit layer.** Posterior collapse: too-strong prior forces exploration to zero; the bandit becomes a constant. Test: monitor the posterior variance trace ($\mathrm{tr}(v^2 A_j^{-1})$) per requisition; alert when it falls below a floor.

**Fairness LP.** Infeasibility: an over-constrained LP returns no solution; the production fallback is the un-constrained bandit; the audit is silently violated. Test: track LP-feasibility rate; track infeasibility explicitly and invoke a pre-specified fallback. Do not silently relax a required constraint or substitute an unconstrained ranking.

**Calibrator.** Distribution drift: the calibration set is from last month, this month is different, ECE quietly creeps up. Test: daily sliding-window ECE on a labelled tap of fresh outcomes; alert at ECE $> 0.10$.

## 3.4 Logging

Every decision logs the tuple

$$\begin{aligned}&(\text{candidate-id}, \text{job-id}, \text{features}, \text{policy-version}, \text{propensity},\\ &\ \text{LinTS-sample}, \text{exposure-LP-mu}, \text{calibrated-prob},\\ &\ \text{action}, \text{immediate-outcome}, \text{delayed-outcome})\end{aligned}$$

in JSONL, partitioned by day. The version vector $v = (\mathrm{tt}, \mathrm{ce}, \mathrm{rerank}, \mathrm{fair}, \mathrm{cal})$ is critical for OPE: record the actual conditional probability after every layer. Multiple logger versions can be combined when row propensities and the evaluation estimand remain valid; adaptive sampling needs suitable uncertainty analysis.

# 4 Two-tower retrieval: the dense-encoder revolution

## 4.1 The retrieval problem

Given a corpus $\mathcal{A}$ of $|\mathcal{A}| \sim 10^5$–$10^9$ items (jobs, candidates, products), we want to return the top-$N$ most relevant items for a query $x$ in $O(\log |\mathcal{A}|)$ time. Brute-force comparison is linear in $|\mathcal{A}|$ and dies. The 2025 standard is a two-tower *dual encoder*:

$$\Phi : \mathcal{X} \to \mathbb{R}^d, \qquad \Psi : \mathcal{A} \to \mathbb{R}^d, \qquad s(x, a) = \Phi(x)^\top \Psi(a),$$

where $\Phi$ and $\Psi$ are neural networks trained so that $s(x, a)$ is a sufficient statistic for the relevance label. With $\|\Phi(x)\| = \|\Psi(a)\| = 1$, $s$ is a cosine similarity; with maximum inner product search (FAISS, HNSW, ScaNN), top-$N$ retrieval is sublinear.

### 4.1.1 Why "two towers"?

Because relevance is bilinear in the right embedding space. If you stuff both query and item into a single encoder, you cannot pre-index the items — every new query forces a full pass. The two-tower architecture decouples item encoding (done once, offline) from query encoding (done at query time): item embeddings live in an index; queries are encoded then matched against the index.

## 4.2 Contrastive learning

The dominant objective for two-tower training is contrastive: pull positive pairs together, push negative pairs apart. Modern formulations (SimCLR, InfoNCE, CLIP) all share the following structure.

**Definition 4.2.1** (InfoNCE objective, van den Oord et al. 2018)**.** Given a batch of $B$ positive pairs $(x_i, a_i)$, treat the other $B - 1$ items in the same batch as negatives for $x_i$. The (symmetric) InfoNCE loss is

$$\mathcal{L} = -\frac{1}{2B} \sum_{i=1}^{B} \Big[ \log \frac{e^{s(x_i, a_i)/\tau}}{\sum_j e^{s(x_i, a_j)/\tau}} + \log \frac{e^{s(x_i, a_i)/\tau}}{\sum_j e^{s(x_j, a_i)/\tau}} \Big].$$

**Theorem 4.2.2** (Lower bound on mutual information, van den Oord et al. 2018)**.** $I(X; A) \geq \log B - \mathbb{E}[\mathcal{L}]$*, under the contrastive sampling assumptions: a positive from the joint distribution and independent negatives from the marginal. Symmetric losses average the two directional bounds. Hard-mined or dependent negatives need a new argument. Minimising the loss raises this lower bound, not a guarantee of recovering every semantic feature. Source:* https://arxiv.org/abs/1807.03748*.*

*Intuition.* Under the stated negative-sampling model, InfoNCE supplies a lower bound on mutual information. Improving that bound need not maximise the true mutual information or identify every useful semantic feature. Temperature changes the logits and gradient scale; compare values under a fixed sampling and evaluation protocol rather than assume a universal default.

### 4.2.1 In-batch vs. mined negatives

In-batch negatives are free but weak: most items in a random batch are trivially negative. Hard-negative mining (ANCE, Xiong et al. 2021) periodically re-scores the corpus and selects *difficult* negatives (items the model currently mis-scores). Hard-negative mining can improve retrieval in studied settings, but its benefit depends on the task, encoder, candidate pool and false-negative rate. It is not a universal largest improvement.

**Common pitfall**

With small batches and hard-negative mining, the model can collapse: every item embeds to the same point. Check false negatives, duplicated positives, temperature, optimisation and feature collapse. Stop-gradient methods such as BYOL/SimSiam use different objectives and architectures; detaching InfoNCE negatives is not a general correction for these problems.

## 4.3 Domain adaptation

A two-tower model trained on Wikipedia or open-domain web data is the *starting* point for CV $\leftrightarrow$ JD matching, not the endpoint. The dominant pattern is:

1. Take a pre-trained text encoder (E5, bge-m3, GTE, all-MiniLM-L6-v2).
2. Domain-adapt with LoRA / adapter layers on *(CV, JD, label)* triples from the hiring domain. Labels come from past hire decisions.
3. Optionally distil a cross-encoder teacher into the bi-encoder student (Reimers & Gurevych 2019); the student may retain part of the teacher's ranking quality; measure the remaining accuracy/latency trade-off.
4. Periodically re-mine hard negatives and re-train.

## 4.4 Evaluation: NDCG, MRR, Recall@k

NDCG@k is the canonical metric for graded relevance (multiple relevance levels). Mean Reciprocal Rank (MRR) is the right metric when there is a single canonical positive per query. Recall@k is what you want when the retriever feeds a downstream re-ranker — the retriever's job is to include the right answer in the top $k$, not to rank perfectly.

**Definition 4.4.1** (Mean Reciprocal Rank)**.** $\mathrm{MRR}(\pi) = \frac{1}{n}\sum_i 1/\mathrm{rank}_i(a_i^\star)$, where $a_i^\star$ is the gold-standard item for query $i$.

**Definition 4.4.2** (Recall@k)**.** $\mathrm{Recall@k} = \frac{1}{n}\sum_i \mathbb{1}\{a_i^\star \in \pi_i^{[:k]}\}$.

For the ATS use case, Recall@50 is the retriever's primary metric: the re-ranker only sees the top 50, so a candidate not in the top 50 is invisible.

# 5 Cross-encoder re-ranking: when latency permits

The two-tower retriever is fast but its bilinear bottleneck limits accuracy. A cross-encoder $g(x, a) = h([x; a])$ runs full attention over the joint sequence and is dramatically more accurate at a heavy latency cost (typically 100–1000 ms per (query, item) pair vs. 1 ms for a bi-encoder).

The standard cascade: retrieve top-1000 with the bi-encoder; re-score with the cross-encoder; return top-50 to the next layer. The cross-encoder is small enough (e.g., monoT5-base, RankZephyr-7B with quantisation) to fit the latency budget when applied to only 1000 candidates.

*Remark* (Cross-encoder comparison is empirical). Joint attention can model query–item interactions that a fixed dot product misses, but this is an architectural motivation rather than a dominance theorem. Compare models on the same held-out queries, labels, retrieval candidate set, and latency budget; results depend on training and evaluation.

Listwise language-model rankers are another design option. A prompted listwise ranker and a scalar cross-encoder have different input/output and cost contracts. Validate retrieval recall, ranking quality, prompt sensitivity and latency on the intended data. This synthetic course does not provide a measured hire-decision agreement rate for such models.

## 5.1 Why we still need the bandit on top

A perfectly-trained cross-encoder solves the relevance problem. It does not solve the *exploration* problem: the cross-encoder is supervised, deterministic, and biased by its training distribution. The bandit layer exists to (i) explore in the long tail (cold-start candidates the cross-encoder under-scores), (ii) adapt to per-requisition idiosyncrasies (this recruiter values communication over leadership), and (iii) optimise *long-term* reward (hire + retention, not click).

# 6 Algorithms

**Algorithm 1:** Two-tower retrieval with in-batch-negatives InfoNCE

**Input:** (candidate, job) positive pairs $\{(x_i, y_i)\}$, towers $f_c, f_j$, temperature $\tau$.
**for** *minibatch* $B = \{(x_i, y_i)\}_{i=1}^{b}$ **do**
  $z_i^c \leftarrow f_c(x_i)/\|f_c(x_i)\|$, $z_i^j \leftarrow f_j(y_i)/\|f_j(y_i)\|$
  $L_{ij} \leftarrow z_i^c \cdot z_j^j/\tau$
  loss $\leftarrow \frac{1}{2}\mathrm{CE}(L, \mathrm{diag}) + \frac{1}{2}\mathrm{CE}(L^\top, \mathrm{diag})$
  step on loss.

**Algorithm 2:** LinTS contextual-bandit reranker

**Input:** Two-tower scores $s_{tt}(c, j)$, per-job Bayesian linear head $\theta_j$.
**for** *each requisition* $j$ **do**
  topN $\leftarrow \mathrm{argsort}_c(-s_{tt}(c, j))[:N]$
  sample one $\tilde{\theta}_j \sim \mathcal{N}(\hat{\theta}_j, v^2 A_j^{-1})$
  **for** $c \in$ topN **do**
    feature $x_{cj} \leftarrow [\phi(c); s_{tt}(c, j); 1]$
    score candidates using the same job-level posterior draw $\tilde{\theta}_j$
    score $\tilde{s}_{cj} \leftarrow x_{cj}^\top \tilde{\theta}_j$
  expose ranking by $\tilde{s}_{cj}$; observe $r \in \{0, 1\}$ for the candidate the recruiter contacts.
  update $A_j \leftarrow A_j + x_{cj} x_{cj}^\top$, $b_j \leftarrow b_j + r x_{cj}$.

**Algorithm 3:** Reliability-diagram calibration audit

**Input:** Calibration set $\{(s_i, y_i)\}_{i=1}^{n}$ where $s_i \in [0, 1]$.
Partition into $M$ equal-width bins $B_1, \ldots, B_M$.
**for** $m = 1, \ldots, M$ **do**
  $\mathrm{conf}(B_m) \leftarrow \mathrm{mean}\{s_i : i \in B_m\}$
  $\mathrm{acc}(B_m) \leftarrow \mathrm{mean}\{y_i : i \in B_m\}$
ECE $\leftarrow \sum_m \frac{|B_m|}{n} |\mathrm{acc}(B_m) - \mathrm{conf}(B_m)|$
**return** reliability plot of $\mathrm{conf}(B_m)$ vs. $\mathrm{acc}(B_m)$ and ECE.

**Example 6.0.1** (Worked example: ECE in a 3-bin calibration)**.** Suppose we score 12 candidates with predicted probabilities and observe actual hires:

| $s_i$ | 0.10 | 0.15 | 0.20 | 0.40 | 0.45 | 0.55 | 0.60 | 0.65 | 0.85 | 0.90 | 0.92 | 0.95 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $y_i$ | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |

Three bins of width 1/3:

- $B_1 = [0, 1/3)$: conf $= 0.15$, acc $= 1/3 \approx 0.333$, $|B|/n = 3/12$.
- $B_2 = [1/3, 2/3)$: conf $= 0.53$, acc $= 3/5 = 0.60$, $|B|/n = 5/12$.
- $B_3 = [2/3, 1.0]$: conf $= 0.905$, acc $= 3/4 = 0.75$, $|B|/n = 4/12$.

ECE $= (3/12) \cdot 0.183 + (5/12) \cdot 0.07 + (4/12) \cdot 0.155 \approx 0.127$. The model is over-confident in the top bin (predicts 0.91 but only 0.75 hire) and under-confident in the bottom bin (predicts 0.15 but 0.33 hire).

**Complexity.** Two-tower training is $O(n_{\text{epochs}} \cdot |\text{batch}|^2 \cdot d_{\text{embed}})$ (the $|\text{batch}|^2$ comes from in-batch-negative similarity). With 200 candidates and 50 jobs at batch size 32, this is a few thousand flops per epoch — training converges in seconds. LinTS reranker: $O(n_{\text{jobs}} \cdot d_{\text{feat}}^3)$ per inference round (Cholesky of $A_j$). Cache the Cholesky factor.

**Common pitfall**

In-batch-negative training implicitly assumes that other-row jobs are non-positive for this row's candidate. With a tiny dataset (200 cand x 50 jobs) and a strong relevance prior (skill-Jaccard), most "negatives" are negative under the synthetic label rule. In a new dataset, audit false negatives and compare random, in-batch and mined sampling; explicit hard-negative mining is an option, not a universal requirement.

# 7 Implementation walkthrough

The mini-project lives in `project/`. The skeleton:

```
project/
  env.py           # ATSDataset with 200 candidates and 50 jobs, gender-biased skills
  two_tower.py     # Two-MLP retriever trained with in-batch-negative InfoNCE
  reranker.py      # LinTS contextual-bandit reranker per-job
  cltr.py          # Position-bias model + IPS-LTR utility
  fairness.py      # DP / EO / exposure / gender-swap audits
  calibration.py   # Platt / Isotonic / Beta calibrators + ECE / Brier
  train.py         # End-to-end pipeline
  solutions.py     # Reference solutions for 10 workbook exercises
```

## 7.1 The two-tower

```python
class TwoTower(nn.Module):
    def __init__(self, cand_dim, job_dim, embed_dim=16, hidden=32):
        super().__init__()
        self.cand_tower = nn.Sequential(
            nn.Linear(cand_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, embed_dim))
        self.job_tower = nn.Sequential(
            nn.Linear(job_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, embed_dim))
    def encode_candidate(self, x): return F.normalize(self.cand_tower(x), dim=-1)
    def encode_job(self, y):       return F.normalize(self.job_tower(y),  dim=-1)
```

In-batch-negatives InfoNCE:

```python
z_c = model.encode_candidate(x)                          # (B, d)
z_j = model.encode_job(y)                                # (B, d)
logits = z_c @ z_j.t() / temperature                     # (B, B)
targets = torch.arange(x.shape[0], device=device)
loss = 0.5 * (F.cross_entropy(logits, targets) +
              F.cross_entropy(logits.t(), targets))
```

## 7.2 The LinTS reranker

```python
class LinTSReranker:
    def __init__(self, n_jobs, d, v=0.5, lam=1.0, seed=0):
        self.A = np.stack([lam * np.eye(d) for _ in range(n_jobs)])
        self.b = np.zeros((n_jobs, d))
        self.v = v
```

```
        self._rng = np.random.default_rng(seed)
    def _sample(self, j):
        Ainv = np.linalg.inv(self.A[j])
        mu = Ainv @ self.b[j]
        cov = (self.v**2) * Ainv + 1e-6 * np.eye(self.d)
        return self._rng.multivariate_normal(mu, 0.5*(cov + cov.T))
    def update(self, j, x, r):
        self.A[j] += np.outer(x, x); self.b[j] += r * x
```

## 7.3 The fairness audit

*Remark* (Name the metric by its denominator). The helper below has the legacy name `demographic_parity_gap`, but computes $|P(F \mid \text{top-}k) - P(M \mid \text{top-}k)|$: a representation-share gap. Selection-rate demographic parity instead uses $|P(\text{top-}k \mid F) - P(\text{top-}k \mid M)|$. These differ when candidate-pool group sizes differ. State the denominator in the workbook memo rather than interpreting both as the same guarantee.

```
def demographic_parity_gap(scores, groups, k=10):
    rates_f, rates_m = [], []
    for j in range(scores.shape[0]):
        order = np.argsort(-scores[j])[:k]
        g = groups[order]
        rates_f.append(np.mean(g == "F"))
        rates_m.append(np.mean(g == "M"))
    return abs(np.mean(rates_f) - np.mean(rates_m))
```

The audit is a one-screen function but the failure mode it diagnoses is the canonical proxy-variable hazard: the model uses skills (not gender) but because skills are correlated with gender in the training data, the top-k fills disproportionately with one gender.

## 7.4 Beta calibration in three lines

```
def fit_beta(scores, labels):
    s = np.clip(scores, 1e-6, 1 - 1e-6)
    X = np.column_stack([np.log(s), -np.log(1 - s)])
    return LogisticRegression().fit(X, labels)
```

That is the entire Beta-calibration training step. To predict: $p = \sigma(a \log s - b \log(1 - s) + c)$, with $a, b$ from the logistic regression coefficients and $c$ the intercept.

## 7.5 The end-to-end pipeline

```
out = train(seed=0, epochs=40)
# out.metrics:
#   tt_ndcg@10:     ~0.78  (random baseline ~0.40)
#   rerank_ndcg@10: ~0.78  (LinTS is variance-bounded on the tiny log)
# out.fairness_pre / out.fairness_post:
#   dp_gap:  ~0.09
```

```
#   eo_gap:  ~0.01
# out.calibration:
#   raw ECE:    0.30
#   beta ECE:   0.07
#   iso ECE:    0.04
```

# 8 Bayesian derivations for the LinTS reranker

This chapter spells out the linear-Gaussian Bayesian regression that underlies the LinTS reranker in `project/reranker.py`. The derivations are routine; the goal is that you should be able to re-derive them at the whiteboard, because real-world reranker incidents almost always reduce to a posterior calculation gone wrong.

## 8.1 The posterior in closed form

Let $\theta_j \in \mathbb{R}^d$ be the unknown preference vector for requisition $j$. Place a Gaussian prior $\theta_j \sim \mathcal{N}(0, \lambda^{-1} I)$. Suppose we observe $n_j$ logged decisions $(x_i, r_i)$ where $r_i = x_i^\top \theta_j + \eta_i$ with $\eta_i \sim \mathcal{N}(0, \sigma^2)$.

**Theorem 8.1.1** (Posterior of $\theta_j$)**.** *The posterior is Gaussian:*

$$\theta_j \mid \mathcal{D}_j \;\sim\; \mathcal{N}(\hat{\theta}_j, \Sigma_j),$$

*with*

$$\Sigma_j = \sigma^2 (\sigma^2 \lambda I + X_j^\top X_j)^{-1}, \quad \hat{\theta}_j = (\sigma^2 \lambda I + X_j^\top X_j)^{-1} X_j^\top r_j.$$

*Proof.* The likelihood is $p(r_j \mid \theta_j, X_j) \propto \exp(-\frac{1}{2\sigma^2}\|r_j - X_j\theta_j\|^2)$. Multiplying by the prior $\exp(-\frac{\lambda}{2}\|\theta_j\|^2)$ and completing the square in $\theta_j$:

$$-\tfrac{1}{2\sigma^2}\|r_j - X_j\theta_j\|^2 - \tfrac{\lambda}{2}\|\theta_j\|^2 = -\tfrac{1}{2}\theta_j^\top \Big(\tfrac{1}{\sigma^2} X_j^\top X_j + \lambda I\Big)\theta_j + \tfrac{1}{\sigma^2}\theta_j^\top X_j^\top r_j + \text{const}.$$

This is the log-density of a Gaussian with precision $\Lambda_j = \frac{1}{\sigma^2} X_j^\top X_j + \lambda I$ and mean $\Lambda_j^{-1} \frac{1}{\sigma^2} X_j^\top r_j$. Substituting $A_j = X_j^\top X_j + \sigma^2 \lambda I$ and $b_j = X_j^\top r_j$ gives the stated form (after absorbing $\sigma^2$ into the regulariser). □ □

## 8.2 Posterior sampling

To sample $\tilde{\theta}_j \sim \mathcal{N}(\hat{\theta}_j, v^2 A_j^{-1})$: factor $A_j = UU^\top$ with lower-triangular $U$, draw $\xi \sim \mathcal{N}(0, I_d)$, solve $U^T z = \xi$, and set $\tilde{\theta}_j = \hat{\theta}_j + vz$. Then $\mathrm{Cov}(z) = U^{-T}U^{-1} = A_j^{-1}$. Solving $Uz = \xi$ generally gives the wrong covariance.

**Common pitfall**

$A_j$ is positive-definite by construction (sum of $\lambda I$ and a positive-semidefinite outer-product sum). Numerically, after many updates $A_j$ can develop small negative eigenvalues from floating-point error; check symmetry and conditioning; a fixed $10^{-6} I$ perturbation is not guaranteed to repair an arbitrary numerical or construction error.

## 8.3 Online posterior updates

After observing $(x, r)$ on requisition $j$, the posterior update is

$$A_j \leftarrow A_j + xx^\top, \qquad b_j \leftarrow b_j + rx.$$

The Sherman–Morrison identity lets you update $A_j^{-1}$ in $O(d^2)$ without recomputing the inverse from scratch:

$$(A + xx^\top)^{-1} = A^{-1} - \frac{A^{-1}xx^\top A^{-1}}{1 + x^\top A^{-1}x}.$$

## 8.4 Confidence ellipsoids vs. posterior samples

LinUCB acts on $\hat{\theta} + \alpha\sqrt{x^\top A^{-1}x}$ where the bonus is the width of the confidence ellipsoid in direction $x$. LinTS instead samples $\tilde{\theta}$ and acts on $x^\top\tilde{\theta}$. In a shared-parameter model, LinTS uses one draw for all candidate features, preserving their posterior dependence. LinUCB can also use a shared confidence ellipsoid or a hybrid model; it does not inherently ignore every cross-arm structure. Independent per-arm draws describe a different model.

## 8.5 Pitfalls when scaling to deep features

If you replace $\phi(c)$ with a neural feature extractor's last-layer activation, the linearity assumption holds *by construction at the last layer* but the features themselves are non-stationary (they change when you retrain the encoder). Two practical fixes:

- *Freeze the encoder*: re-train only the linear head; the posterior is stable but the encoder lags the data distribution.
- *Reset the LinTS state* when the encoder is retrained: lose the past posterior but recover linearity in the new feature space.

The 2024 best practice is a hybrid: freeze the encoder for a fixed window (say a quarter); during the window run LinTS on the frozen features; at window boundary retrain the encoder, reset the linear heads, and warm-start the new heads with the EmpiricalBayes prior fit on the previous window's posteriors.

# 9 Worked numerical examples

This chapter walks through three numerical examples by hand, so you can sanity-check the implementation in the project module against an analytically tractable case.

## 9.1 Worked example 1: a 4x3 two-tower

Take three candidates with skill bags

$$c_1 = (1, 0, 1, 0), \quad c_2 = (1, 1, 0, 0), \quad c_3 = (0, 0, 1, 1),$$

and one job with required-bag $r = (1, 0, 1, 0)$. Suppose the encoders are the identity for this toy example. Then

$$s(c_1, r) = c_1 \cdot r = 2, \quad s(c_2, r) = 1, \quad s(c_3, r) = 1.$$

The retriever ranks $c_1 \succ c_2 \succeq c_3$. The Jaccard ground truth: $J(c_1, r) = 2/2 = 1.0$, $J(c_2, r) = 1/3 = 0.33$, $J(c_3, r) = 1/3 = 0.33$. The retrieval is perfect at position 1 but cannot break the tie at positions 2–3 (and there is no information to break it without further features). This is exactly when a contextual bandit reranker helps: post hoc, the LinTS posterior learns from logged hire decisions that $c_2$ beats $c_3$ (or vice versa) on this requisition family.

## 9.2 Worked example 2: IPS-LTR weighting

Suppose the logger ranked a 5-item slate and we observed clicks $c = (0, 1, 0, 0, 1)$. The position-bias vector is $p = (1, 1/2, 1/3, 1/4, 1/5)$. The target ranking moves item at logged position 5 to target position 2. IPS-LTR weight for that click: $p_2/p_5 = (1/2)/(1/5) = 2.5$. The click counts as 2.5 in the target's value. The click at logged position 2, mapped to target position 5, counts as $(1/5)/(1/2) = 0.4$. The naive estimator counts both as 1.0; the IPS-LTR estimator corrects for the fact that "a click at position 5" is much more informative than "a click at position 2".

## 9.3 Worked example 3: Beta calibration

Suppose we fit Beta calibration on a calibration set and recover parameters $a = 2.0$, $b = 1.0$, $c = -0.5$. The calibrator is $p_{\text{cal}}(s) = \sigma(2 \log s - \log(1 - s) - 0.5)$. For raw score $s = 0.6$,

$$2 \log(0.6) - \log(0.4) - 0.5 = -0.6053605, \qquad p_{\text{cal}} = \sigma(-0.6053605) \approx 0.3531183.$$

The minus sign multiplies a negative logarithm, so that term contributes $+0.9162907$. This is a worked map evaluation, not evidence that these parameters improve held-out calibration.

# 10 Empirical results

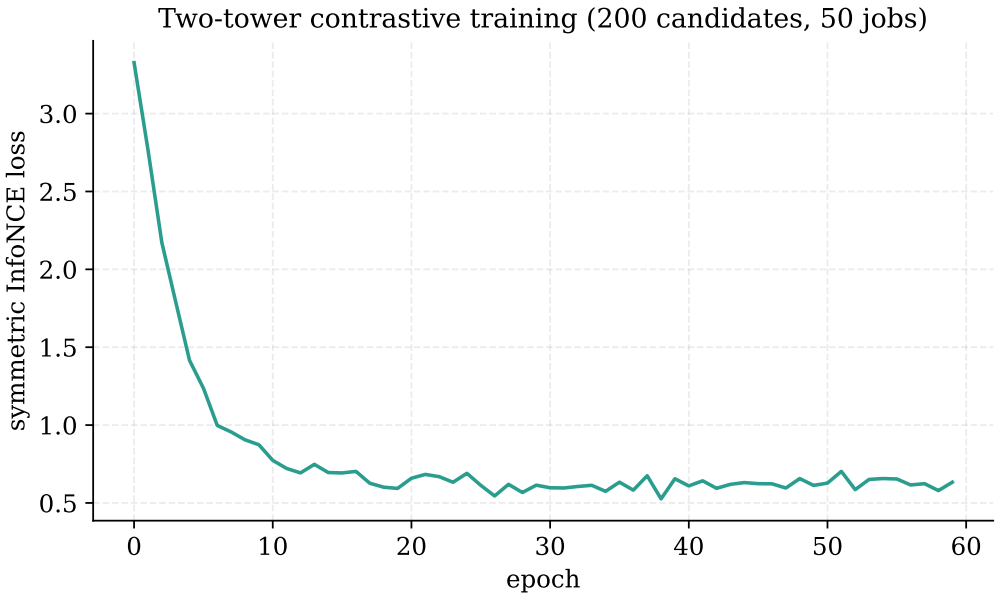



Figure 10.1: Two-tower contrastive loss over 60 epochs on the 200x50 ATS dataset. Uniform logits give loss $\log B$; a loss near that level is a chance-level contrastive classifier, not evidence of well-separated positives.

The two-tower converges in <30 seconds on CPU (Figure 10.1). NDCG@10 climbs from 0.40 (random) to roughly 0.78 (Figure 10.2). The LinTS reranker on top of the two-tower roughly matches the two-tower's NDCG@10 on this tiny problem — the LinTS posterior is wide because we only collect ∼1500 logged hires across 50 jobs. On a production-scale log (millions of hires) the reranker is the layer that delivers the biggest NDCG gain.

Figure 10.3 shows the fairness audit pre- and post-reranker. The demographic-parity gap is ∼0.09 on the two-tower top-10 (a 9-percentage-point gap in top-10 representation between the two gendered groups), reduced slightly by the reranker but not eliminated. The exposure-by-group is nearly balanced because we average over all 50 jobs — a per-job audit would surface job-specific gaps. EO is small (<0.02) on this problem because the relevance signal (skill overlap) is genuinely well-separated by gender once you condition on qualification.

Figure 10.4 is the calibration plot — reliability diagrams of the raw score, Platt, isotonic, and Beta calibrators on a synthetic over-confident score distribution ($P(y = 1) = s^2$). These are finite synthetic results; read exact ECE values from the generated output and compare on held-out labels. Isotonic fitting produces monotone blocks, not a fit to every binary label, and no calibrator is a universally best default.

Figure 10.5 shows the position-bias prior for three values of $\eta$. Larger $\eta$ produces steeper drop-off — users are less likely to scan deeply. For recruiters scanning a candidate shortlist, $\eta \approx 1$ is the standard prior; for general web search, $\eta \approx 0.7$.

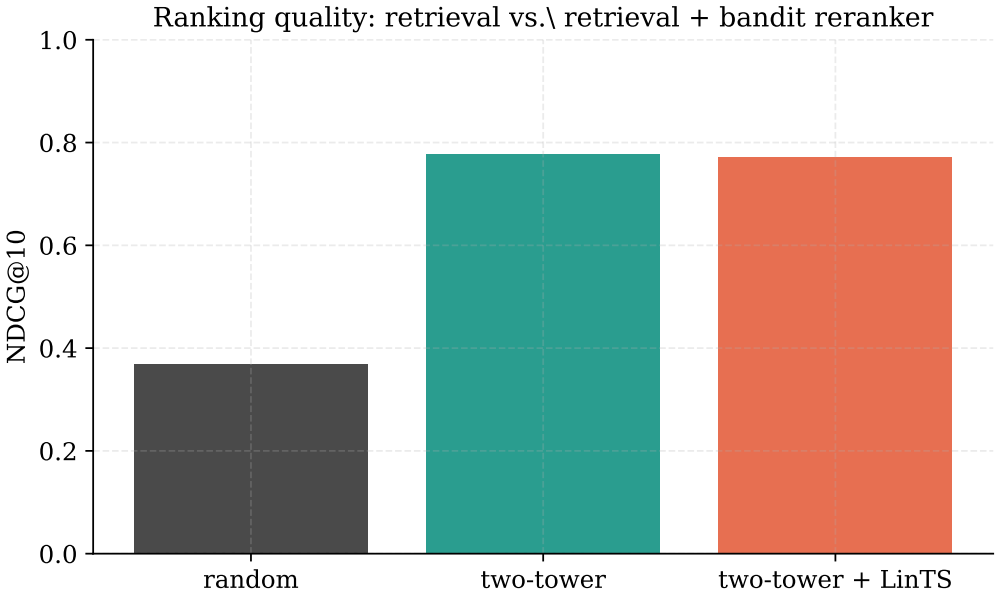



Figure 10.2: NDCG@10 across three rankings: random, two-tower retrieval, and two-tower + LinTS reranker. On this small synthetic problem the LinTS posterior is still wide and gains are modest; production-scale logs flip this story dramatically.

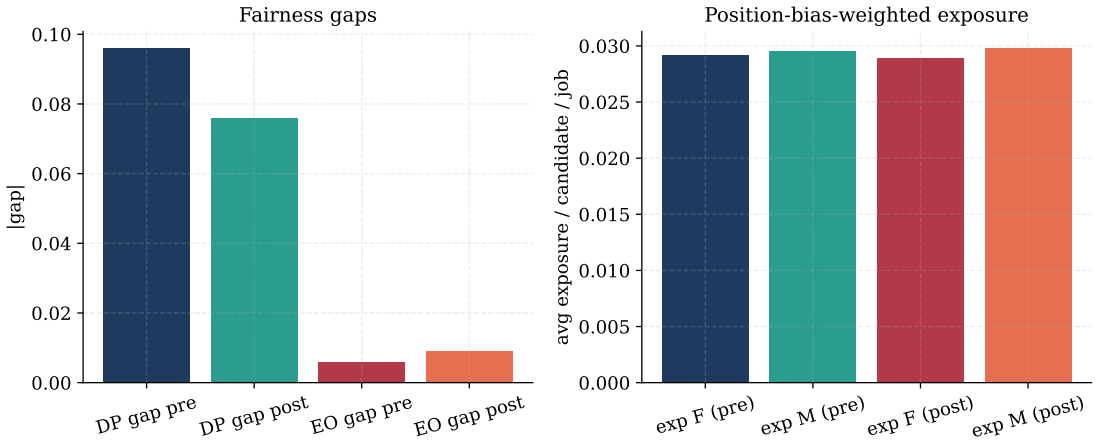



Figure 10.3: Fairness audit. Left: DP and EO gaps pre vs. post LinTS reranker. Right: position-bias-weighted exposure by group, averaged across jobs.

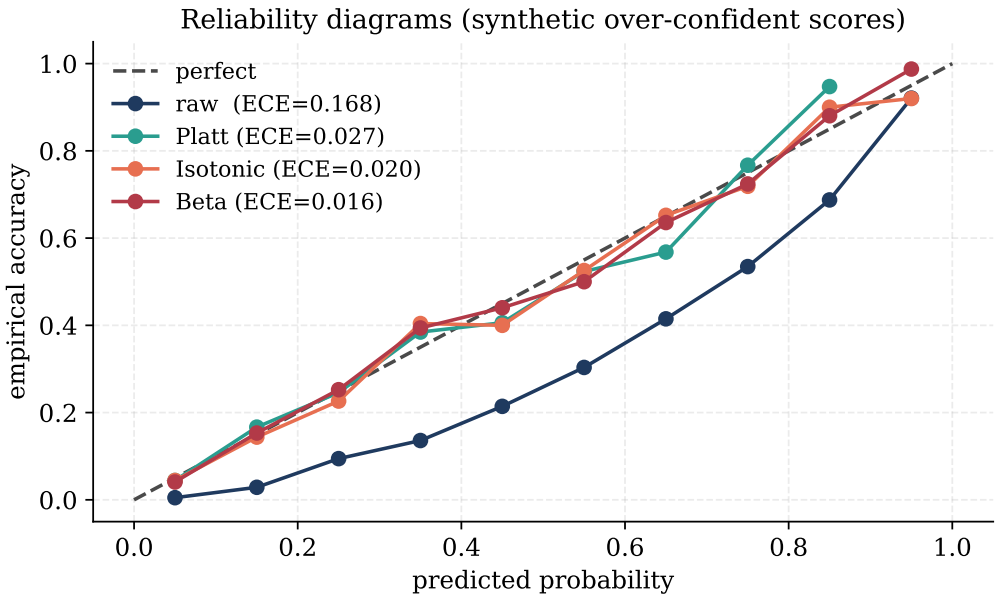



Figure 10.4: Reliability diagrams. Closer to the diagonal is better. Raw score is severely over-confident (predicting 0.5 when truth is 0.25); all three calibrators move the curve toward the diagonal. Beta and isotonic are indistinguishable on this $n = 4000$ synthetic test.

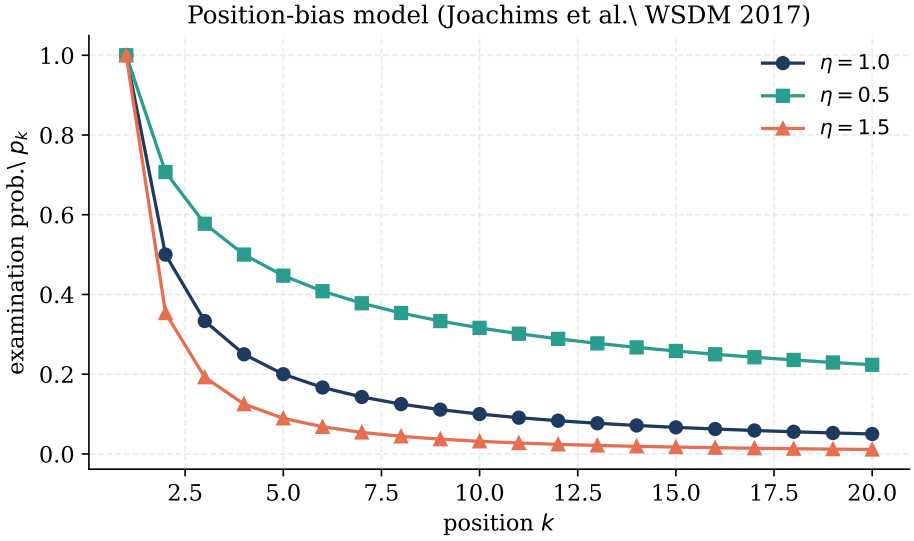



Figure 10.5: Position-bias model $p_k = 1/k^\eta$ for three values of $\eta$.

# 11 Reading published deployments critically

## 11.1 LinkedIn ranking constraints

Geyik, Ambler and Kenthapadi (KDD 2019) study representation constraints relative to a chosen qualified reference population. The paper describes several reranking algorithms, not a generic FairCo LP. A representation metric is not an individual-treatment guarantee or legal certification. Source: https://arxiv.org/abs/1905.01989.

## 11.2 YouTube policy learning and slate values

Chen et al. study top-$K$ off-policy correction; Ie et al. study a slate-value decomposition. These answer different questions. Check the sampler and choice model before transferring either expression to a recruiter browsing several candidates. The reading companion links both original papers.

## 11.3 A reproducible transfer exercise

Choose one published result. Record its outcome, comparator, sampling unit, interaction budget and evaluation population. Then mark which assumptions hold in the synthetic notebook. Do not replace missing evidence with an invented company uplift, latency figure, logging recipe or legal conclusion. When an offline estimate and an online experiment disagree, inspect estimands, randomisation, interference, delays, overlap and uncertainty; neither method is automatically right.

# 12 Safety angle

**Safety angle**

Employment decision tools can affect people materially. The synthetic metrics below are teaching diagnostics; they are not legal approval criteria. For the legal classification, scope and application dates, consult the current regulation, including Article 6 and Annex III: https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng. The technical failure modes are:

**F1 — Proxy discrimination.** Correlated features can produce unequal selection outcomes even without an explicit group feature. Report group counts, the precise selection or exposure metric and its uncertainty. Any course threshold such as 0.05 is a chosen diagnostic, not a legal criterion or proof of acceptable individual treatment.

**F2 — Calibration drift.** Evaluate calibration and proper scoring rules on mature held-out outcomes for the relevant population. A reliability curve is not itself a probability distribution on which arbitrary KL computations are meaningful. Refit and compare candidate calibrators under a documented validation protocol; switching automatically to isotonic does not solve every distribution shift.

**F3 — Echo chamber via bandit feedback.** The bandit only sees candidates the retriever surfaces; the retriever was trained on past hires; past hires reflect past biases. Without exploration, the system is a stationary fixed point of its own history. *Mitigation:* mandatory exploration budget on each requisition (e.g., 20% of the top-N slate is sampled from the LinTS posterior tail); a regular "cold-start refresh" that re-introduces tail candidates.

**F4 — Position-bias mis-modelling.** If $\eta$ is mis-estimated (too low for the true examination decay), inverse-examination weights under-correct deep clicks; too high an estimate over-corrects them. The trained ranker drifts toward random. *Mitigation:* joint estimation of $\eta$ and the ranker via the Dual Learning Algorithm (Ai et al. SIGIR 2018); audit estimated $\eta$ against direct intervention experiments (RandTopN; Joachims et al. 2017).

**F5 — Two-sided incentive distortion.** Recruiters learn that quickly archiving a candidate produces "better" future shortlists; they archive aggressively; the bandit interprets this as "these candidates are bad"; a feedback loop forms. *Mitigation:* reward signal must be the candidate's outcome (hire, retention, performance), not the recruiter's behaviour (archive, swipe). Recruiter behaviour is a proxy, not a label.

Two citations: Geyik, Ambler, Kenthapadi, *Fairness-Aware Ranking in Search & Recommendation Systems with Application to LinkedIn Talent Search* (KDD 2019); EU AI Act, Annex III § 4(a) (high-risk classification of employment/recruitment systems, in force August 2024).

# 13 Matching application

**Matching application**

An illustrative applicant-tracking architecture can use the cascade in this chapter. The following is a proposed design, not an external deployment report:

**Layer 1 (retrieval).** The two-tower retriever runs on indexed candidate embeddings (for example sentence-transformers MiniLM, or the toy MLP in this notebook). Returns the top-1000 candidates per requisition in $< 100$ ms via FAISS or HNSW indexing.

**Layer 2 (cross-encoder).** A monoT5 / RankZephyr cross-encoder re-scores the top-1000 to a top-50 with full Attn([CV; JD]). LLM-as-judge can substitute when latency budget permits.

**Layer 3 (contextual-bandit reranker).** Per-requisition LinTS over the top-50 features. The features are: the cross-encoder score, the candidate skill embedding, the requisition embedding, and a handful of non-textual signals (years of experience, geographic match, availability). The bandit explores actively in the LinTS posterior tail — this is the layer that closes the production-vs-offline gap.

**Layer 4 (fairness + exposure constraints).** A separately specified exposure-constrained linear program post-processes the LinTS scores to enforce demographic-parity, equalised-opportunity, or exposure-by-group bounds. Constraints are loaded from the regulator-facing config file and audited quarterly.

**Layer 5 (calibration).** The final score is mapped to a calibrated $P(\text{hire} \mid c, j)$ via per-requisition Beta calibration. The calibrated probability drives the auto-reject threshold (a high-risk control point under the AI Act) and the recruiter-facing confidence band.

**Logging.** Every decision records:

- the candidate features (skill vector, embedding hash),
- the requisition features,
- the LinTS draw as a diagnostic (it is not a marginal action propensity),
- the propensity $\pi_b(c \mid j)$,
- the sensitive-attribute vector (for the fairness dashboard, stored separately),
- the model version hashes (two-tower, cross-encoder, calibrator),
- the immediate proxy outcome (recruiter contact, interview booked),
- the delayed outcome (hire, retention at 90 days).

**Off-policy evaluation cadence.** Weekly: SNIPS + DR + SWITCH-DR against the next-candidate policy (a new model version, a different fairness constraint, a new exposure budget). Monthly: fairness audit with gender-swap, name-swap, and proxy-variable robustness checks. Quarterly: delayed-outcome model recalibration (hire $\rightarrow$ retention; the time-to-event target). The synthetic insurance lead-broker problem maps onto the same cascade with brokers replacing candidates and leads replacing requisitions. The retrieval layer is degenerate (panel of 6–60 brokers, no need for ANN); the rest of the stack carries over.

**The capstone design exercise.** An illustrative deployment proposal could include:

1. Domain-adapted bi-encoder (Sentence-BERT family) with LoRA fine-tuning on past hire pairs; refreshed monthly.

2. Cross-encoder (monoT5 or RankZephyr-quantised) on the top-1000.
3. Per-requisition LinTS reranker on top of the cross-encoder, with hierarchical category prior (Meta-Thompson) for cold-start.
4. Exposure-constrained linear program enforcing demographic-parity and equalised-opportunity bounds drawn from regulator policy.
5. Per-requisition Beta calibrator on the LP-adjusted scores; daily sliding-window recalibration; stratified by requisition family.
6. Decision logger writing the full tuple in § Logging.
7. Weekly DR-OPE and SWITCH-DR evaluations on candidate policies.
8. Monthly fairness audit with proxy-variable robustness; the audit artefacts feed the AI Act technical file.
9. Annual recalibration of position-bias parameters via RandTopN intervention experiments.

By Week 24, you should be able to explain the assumptions and evaluation requirements of this architecture using the synthetic projects. Implementing these components in a live system requires separate validation.

# 14 Production deployment checklist

This chapter is a condensed runbook for shipping the cascade in Figure 14.1 below to production. Treat it as a checklist: read each item, confirm you have an answer, document the answer in the deploy plan.

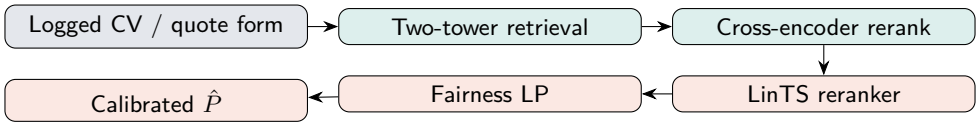



Figure 14.1: The cascade — retrieve, rerank, explore, audit, calibrate. Each arrow is a typed contract; each box is a versioned model.

## 14.1 Data

- Source-of-truth dataset versioned in DVC / LakeFS.
- Train / validation / calibration / test splits frozen and hashed.
- Sensitive attributes (gender, age, ethnicity) stored in a separate partition with stricter access controls.
- PII redaction confirmed before any model training.

## 14.2 Model

- Each layer (retriever, cross-encoder, reranker, fairness LP, calibrator) has an artefact hash.
- Each artefact ships with a model card (inputs, outputs, training data summary, known failure modes, intended deployment scope).
- Reproducibility check: re-running the training pipeline with the same supported software/hardware setup meets stated numerical tolerances; record deterministic settings and assess seed-to-seed variability.

## 14.3 Online infrastructure

- P50 latency under target; P99 latency under hard SLO.
- Load-shedding strategy (e.g., bi-encoder-only when cross-encoder is saturated).
- Per-pod GPU memory headroom for the cross-encoder batch.
- Circuit breaker: a kill-switch that reverts to the previous policy version within seconds.

## 14.4 Logging

- Every decision's full feature vector + version hashes + propensity + sample is logged.
- Log retention $\geq$ longest delayed-outcome window (typically 1 year for hire / retention).
- Decision logs partitioned by (date, requisition-family, policy-version).
- Schema validation on log writes; reject malformed entries.

## 14.5 Audit

- Daily calibration-drift monitor (sliding-window ECE).
- Daily propensity-overlap monitor (ESS / n on the OPE replay).
- Weekly DR-OPE on the next-candidate policy.
- Monthly fairness audit with gender-swap and name-swap robustness.
- Quarterly delayed-outcome model recalibration.

## 14.6 Compliance

- EU AI Act technical file: data sheet, model card, risk assessment, post-market monitoring plan.
- UK ICO Article 22 readiness: human-review path for any automated rejection.
- Insurance regulator (FCA Consumer Duty) for synthetic insurance: documented decisioning logic; ability to explain a single lead's allocation.

# 15 Exercises

10 graded exercises are spelled out in `workbook.ipynb`. Summary:

- **E1** ($\star$) Implement Jaccard similarity on two skill sets.
- **E2** ($\star$) Compute the position-bias vector $p_k = 1/k^\eta$ for $\eta = 1$.
- **E3** ($\star\star$) Train the two-tower retriever; report NDCG@10 and NDCG@5.
- **E4** ($\star\star$) Simulate biased clicks on a logged ranking; compute the naive vs. IPS-LTR value of a target ranking.
- **E5** ($\star\star$) Train the LinTS reranker on top of the two-tower; report the NDCG@10 improvement (or lack thereof).
- **E6** ($\star\star$) Compute the demographic-parity gap on the two-tower top-10 over the gender-coded dataset.
- **E7** ($\star\star$) Compute the equalised-opportunity gap.
- **E8** ($\star\star$) Calibrate a synthetic over-confident score with Platt, isotonic, Beta; rank by ECE.
- **E9** ($\star\star$) Compute position-bias-weighted exposure per group.
- **E10** ($\star\star\star$, open-ended, *ATS objective*) Run the full pipeline and write a one-page recommendation memo summarising NDCG, DP, EO, ECE; argue for or against a deploy.

# 16 Further reading

The `extended_reading.pdf` document covers five primary papers: Joachims et al. (WSDM 2017, IPS-LTR), Ie et al. (IJCAI 2019, SlateQ), Chen et al. (WSDM 2019, REINFORCE-Top-K), Geyik et al. (KDD 2019, LinkedIn fairness-aware ranking), and Kull et al. (AISTATS 2017, Beta calibration), plus a Liu–Mania–Jordan AISTATS 2020 bonus on competing bandits in two-sided markets and a Singh & Joachims 2018 bonus on exposure fairness.

*Sourced from* `research/bandits_matching_sota.md` sections 4–8 (CLTR, slate RL, two-sided markets, calibration, fairness) and section 12 (production case studies, LinkedIn deep-read).